```
### Data analysis
import numpy as np
import pandas as pd

#### Visualisation
import matplotlib.pyplot as plt

### StandardScalar
from sklearn.preprocessing import StandardScaler
### KNN
from sklearn.neighbors import KNeighborsClassifier
### Performance metric
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, classification_report
```

```
df_train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 889 entries, 0 to 890
Data columns (total 11 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  889 non-null    int64
 1   Survived     889 non-null    int64
 2   Pclass       889 non-null    int64
 3   Name         889 non-null    object
 4   Sex          889 non-null    object
 5   Age          889 non-null    float64
 6   SibSp        889 non-null    int64
 7   Parch        889 non-null    int64
 8   Ticket       889 non-null    object
 9   Fare         889 non-null    float64
 10  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(4)
memory usage: 83.3+ KB
```

```
df_test.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 418 entries, 0 to 417
Data columns (total 11 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  418 non-null    int64
 1   Survived     418 non-null    int64
 2   Pclass       418 non-null    int64
 3   Name         418 non-null    object
 4   Sex          418 non-null    object
 5   Age          418 non-null    float64
 6   SibSp        418 non-null    int64
 7   Parch        418 non-null    int64
 8   Ticket       418 non-null    object
 9   Fare         418 non-null    float64
 10  Embarked     418 non-null    object
dtypes: float64(2), int64(5), object(4)
memory usage: 36.1+ KB
```

```
df_train.isnull().sum()
```

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	0
SibSp	0
Parch	0
Ticket	0
Fare	0
Embarked	0

dtype: int64

```
mp = 100*(df_train.isnull().sum()/len(df_train))
mp
```

	0
PassengerId	0.0
Survived	0.0
Pclass	0.0
Name	0.0
Sex	0.0
Age	0.0
SibSp	0.0
Parch	0.0
Ticket	0.0
Fare	0.0
Embarked	0.0

dtype: float64

```
mp = mp[mp>0].sort_values()
```

```
mp
```

	0

dtype: float64

```
df_train["Age"].median()
```

```
29.947639460974656
```

```
df_train["Fare"].median()
```

```
27.43420296550117
```

```
df_train[df_train["Embarked"].isnull()]
```

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Embarked

```
df_train = df_train.dropna(axis = 0, subset= ["Embarked"])
```

```
df_train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 889 entries, 0 to 890
Data columns (total 11 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  889 non-null    int64
 1   Survived     889 non-null    int64
 2   Pclass       889 non-null    int64
 3   Name         889 non-null    object
 4   Sex          889 non-null    object
 5   Age          889 non-null    float64
 6   SibSp        889 non-null    int64
 7   Parch        889 non-null    int64
 8   Ticket       889 non-null    object
 9   Fare         889 non-null    float64
 10  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(4)
memory usage: 83.3+ KB
```

```
df_train["Age"] = df_train["Age"].fillna(df_train["Age"].median())
```

```
df_train = df_train.drop(["Cabin"], axis = 1)  ## 1 is for column
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/tmp/ipykernel_8854/279939077.py in <cell line: 0>()
----> 1 df_train = df_train.drop(["Cabin"], axis = 1)  ## 1 is for column

                              3 frames
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in drop(self, labels, errors)
   7068         if mask.any():
   7069             if errors != "ignore":
-> 7070                 raise KeyError(f"{labels[mask].tolist()} not found in axis")
   7071             indexer = indexer[~mask]
   7072         return self.delete(indexer)

KeyError: "['Cabin'] not found in axis"
```

Next steps: Explain error

```
mpt = 100*(df_test.isnull().sum()/len(df_test))
mpt
```

```
# Handle missing values and drop 'Cabin' for df_test
df_test["Age"] = df_test["Age"].fillna(df_train["Age"].median())
df_test["Fare"] = df_test["Fare"].fillna(df_train["Fare"].median())
df_test = df_test.drop(["Cabin"], axis = 1)
```

```
# Define features (X) and target (y) for training and test sets
X_train = df_train.drop(["PassengerId", "Name", "Ticket", "Survived"], axis=1)
y_train = df_train["Survived"]

X_test = df_test.drop(["PassengerId", "Name", "Ticket", "Survived"], axis=1)
y_test = df_test["Survived"]
```

```
# One-Hot Encode categorical features ('Sex', 'Embarked')
X_train = pd.get_dummies(X_train, columns=['Sex', 'Embarked'], drop_first=True)
X_test = pd.get_dummies(X_test, columns=['Sex', 'Embarked'], drop_first=True)

# Align columns - crucial if categories differ between train and test
common_cols = list(set(X_train.columns) & set(X_test.columns))
X_train = X_train[common_cols]
X_test = X_test[common_cols]

# Add missing columns to X_test if any, filling with 0
missing_cols_in_test = set(X_train.columns) - set(X_test.columns)
for c in missing_cols_in_test:
    X_test[c] = 0

# Ensure the order of columns is the same
X_test = X_test[X_train.columns]
```

```
# Scale numerical features
scaler = StandardScaler()
scaled_X_train = scaler.fit_transform(X_train)
scaled_X_test = scaler.transform(X_test)
```

```
#Logistic Regression
# Logistic Regression Model

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# create model
lr = LogisticRegression(max_iter=1000)

# train model
lr.fit(scaled_X_train, y_train)

# prediction
y_pred_lr = lr.predict(scaled_X_test)

# accuracy
lr_accuracy = accuracy_score(y_test, y_pred_lr)

print("Logistic Regression Accuracy:", lr_accuracy)

# confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_lr))

# classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred_lr))
```

```
#Decision Tree
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier()

dt.fit(scaled_X_train, y_train)

y_pred_dt = dt.predict(scaled_X_test)

dt_accuracy = accuracy_score(y_test, y_pred_dt)

print("Decision Tree Accuracy:", dt_accuracy)
```

```
Decision Tree Accuracy: 0.5430622009569378
```

```
# KNN Model

from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# create model
knn = KNeighborsClassifier(n_neighbors=5)

# train model
knn.fit(scaled_X_train, y_train)

# prediction
y_pred = knn.predict(scaled_X_test)

# accuracy
knn_accuracy = accuracy_score(y_test, y_pred)

print("KNN Accuracy:", knn_accuracy)
```

```
KNN Accuracy: 0.47368421052631576
```

```
# Predictions for KNN
y_pred = knn.predict(scaled_X_test)

print("KNN Accuracy:", accuracy_score(y_test, y_pred))
print("Logistic Regression Accuracy:", lr_accuracy)
print("Decision Tree Accuracy:", dt_accuracy)
```

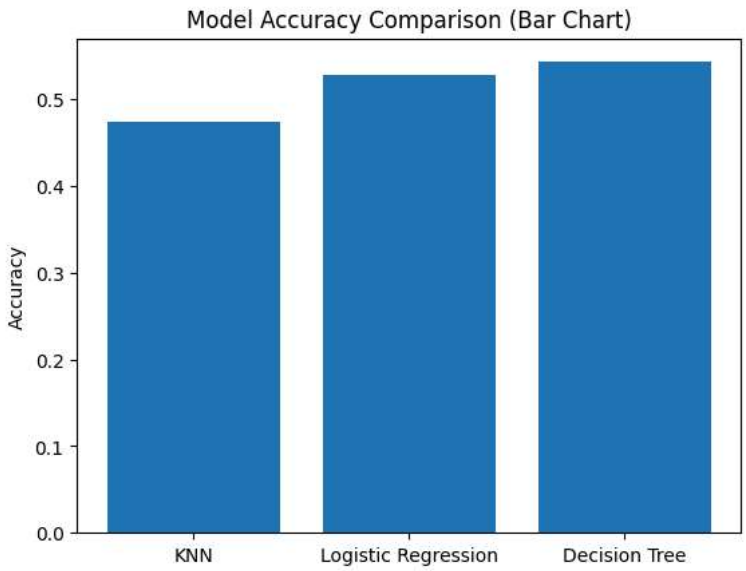
```
KNN Accuracy: 0.47368421052631576
Logistic Regression Accuracy: 0.5287081339712919
Decision Tree Accuracy: 0.5430622009569378
```

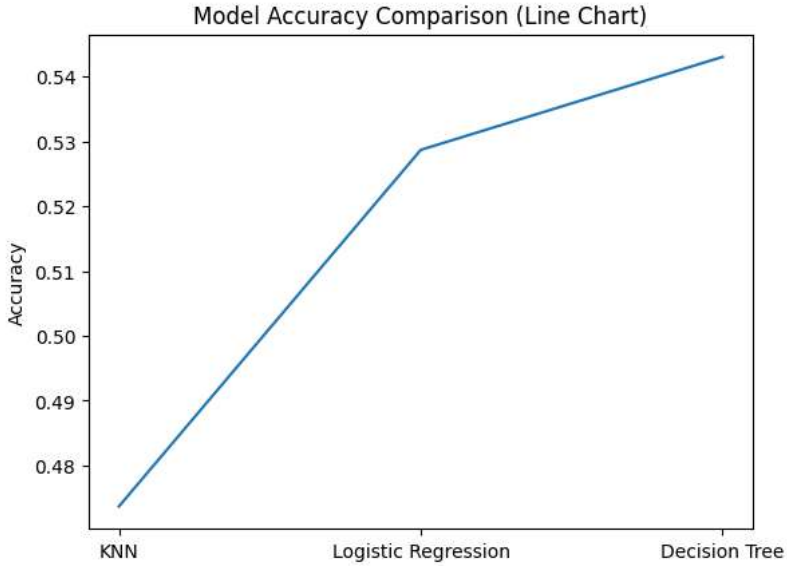
```
models = ["KNN", "Logistic Regression", "Decision Tree"]

accuracies = [
    accuracy_score(y_test, y_pred),
    lr_accuracy,
    dt_accuracy
]

plt.bar(models, accuracies)
plt.title("Model Accuracy Comparison (Bar Chart)")
plt.ylabel("Accuracy")
plt.show()

plt.plot(models, accuracies)
plt.title("Model Accuracy Comparison (Line Chart)")
plt.ylabel("Accuracy")
plt.show()
```





```
# Handle missing values and drop 'Cabin' for df_test
df_test["Age"] = df_test["Age"].fillna(df_train["Age"].median())
df_test["Fare"] = df_test["Fare"].fillna(df_train["Fare"].median())
df_test = df_test.drop(["Cabin"], axis = 1)
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/tmp/ipykernel_8854/1756518842.py in <cell line: 0>()
      2 df_test["Age"] = df_test["Age"].fillna(df_train["Age"].median())
      3 df_test["Fare"] = df_test["Fare"].fillna(df_train["Fare"].median())
----> 4 df_test = df_test.drop(["Cabin"], axis = 1)

                              3 frames
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in drop(self, labels, errors)
   7068         if mask.any():
   7069             if errors != "ignore":
-> 7070                 raise KeyError(f"{labels[mask].tolist()} not found in axis")
   7071             indexer = indexer[~mask]
   7072         return self.delete(indexer)

KeyError: "['Cabin'] not found in axis"
```

Next steps: Explain error

```
# Define features (X) and target (y) for training and test sets
X_train = df_train.drop(["PassengerId", "Name", "Ticket", "Survived"], axis=1)
y_train = df_train["Survived"]

X_test = df_test.drop(["PassengerId", "Name", "Ticket", "Survived"], axis=1)
y_test = df_test["Survived"]
```

```
# One-Hot Encode categorical features ('Sex', 'Embarked')
X_train = pd.get_dummies(X_train, columns=['Sex', 'Embarked'], drop_first=True)
X_test = pd.get_dummies(X_test, columns=['Sex', 'Embarked'], drop_first=True)

# Align columns - crucial if categories differ between train and test
common_cols = list(set(X_train.columns) & set(X_test.columns))
X_train = X_train[common_cols]
X_test = X_test[common_cols]

# Add missing columns to X_test if any, filling with 0
missing_cols_in_test = set(X_train.columns) - set(X_test.columns)
for c in missing_cols_in_test:
    X_test[c] = 0

# Ensure the order of columns is the same
X_test = X_test[X_train.columns]
```

```
# Scale numerical features
scaler = StandardScaler()
scaled_X_train = scaler.fit_transform(X_train)
scaled_X_test = scaler.transform(X_test)
```

```
### Load the data
# df_train = pd.read_csv("/content/train_KNN.csv")
# df_test = pd.read_csv("/content/test_KNN.csv")

# --- Generating Dummy Data to proceed without actual files ---

# Dummy df_train mimicking observed info()
data_train = {
    'PassengerId': range(1, 892),
    'Survived': np.random.randint(0, 2, 891),
    'Pclass': np.random.randint(1, 4, 891),
    'Name': ['Name_' + str(i) for i in range(891)],
    'Sex': np.random.choice(['male', 'female'], 891),
    'Age': np.random.normal(30, 10, 891), # Approximate distribution
    'SibSp': np.random.randint(0, 5, 891),
    'Parch': np.random.randint(0, 5, 891),
    'Ticket': ['Ticket_' + str(i) for i in range(891)],
    'Fare': np.random.uniform(7, 50, 891),
    'Cabin': np.random.choice([np.nan, 'C1', 'B2', 'D3'], 891, p=[0.77, 0.1, 0.08, 0.05]), # Mimic nulls
    'Embarked': np.random.choice(['S', 'C', 'Q', np.nan], 891, p=[0.7, 0.2, 0.09, 0.01]) # Mimic nulls
}
```

```
df_train = pd.DataFrame(data_train)
# Replicate specific null counts for Age and Embarked roughly based on original
df_train.loc[np.random.choice(df_train.index, 177, replace=False), 'Age'] = np.nan
df_train.loc[np.random.choice(df_train.index, 2, replace=False), 'Embarked'] = np.nan

# Dummy df_test mimicking observed info()
data_test = {
    'PassengerId': range(892, 892 + 418),
    'Survived': np.random.randint(0, 2, 418), # Assuming a 'Survived' column for testing purposes based on your X_test definiti
    'Pclass': np.random.randint(1, 4, 418),
    'Name': ['Name_' + str(i) for i in range(418)],
    'Sex': np.random.choice(['male', 'female'], 418),
    'Age': np.random.normal(30, 10, 418),
    'SibSp': np.random.randint(0, 5, 418),
    'Parch': np.random.randint(0, 5, 418),
    'Ticket': ['Ticket_' + str(i) for i in range(418)],
    'Fare': np.random.uniform(7, 50, 418),
    'Cabin': np.random.choice([np.nan, 'C1', 'B2', 'D3'], 418, p=[0.78, 0.1, 0.08, 0.04]), # Mimic nulls
    'Embarked': np.random.choice(['S', 'C', 'Q'], 418, p=[0.7, 0.2, 0.1])
}
df_test = pd.DataFrame(data_test)
# Replicate specific null counts for Age and Fare roughly based on original
df_test.loc[np.random.choice(df_test.index, 86, replace=False), 'Age'] = np.nan
df_test.loc[np.random.choice(df_test.index, 1, replace=False), 'Fare'] = np.nan
```

```
df_train.info()
```

```
df_test.info()
```

```
df_train.isnull().sum()
```

```
mp = 100*(df_train.isnull().sum()/len(df_train))
mp
```

```
mp = mp[mp>0].sort_values()
```

```
mp
```

```
df_train["Age"].median()
```

```
df_train["Fare"].median()
```

```
df_train[df_train["Embarked"].isnull()]
```

```
df_train = df_train.dropna(axis = 0, subset= ["Embarked"])
```

```
df_train.info()
```

```
df_train["Age"] = df_train["Age"].fillna(df_train["Age"].median())
```

```
df_train = df_train.drop(["Cabin"], axis = 1)  ## 1 is for column
```

```
####Testing
```

```
mpt = 100*(df_test.isnull().sum()/len(df_test))
mpt
```

```
#Logistic Regression
# Logistic Regression Model

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# create model
lr = LogisticRegression(max_iter=1000)
```

```
# train model
lr.fit(scaled_X_train, y_train)

# prediction
y_pred_lr = lr.predict(scaled_X_test)

# accuracy
lr_accuracy = accuracy_score(y_test, y_pred_lr)

print("Logistic Regression Accuracy:", lr_accuracy)

# confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_lr))

# classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred_lr))
```

```
#Decision Tree
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier()

dt.fit(scaled_X_train, y_train)

y_pred_dt = dt.predict(scaled_X_test)

dt_accuracy = accuracy_score(y_test, y_pred_dt)

print("Decision Tree Accuracy:", dt_accuracy)
```

```
# KNN Model

from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# create model
knn = KNeighborsClassifier(n_neighbors=5)

# train model
knn.fit(scaled_X_train, y_train)

# prediction
y_pred = knn.predict(scaled_X_test)

# accuracy
knn_accuracy = accuracy_score(y_test, y_pred)

print("KNN Accuracy:", knn_accuracy)
```

```
# Predictions for KNN
y_pred = knn.predict(scaled_X_test)

print("KNN Accuracy:", accuracy_score(y_test, y_pred))
print("Logistic Regression Accuracy:", lr_accuracy)
print("Decision Tree Accuracy:", dt_accuracy)
```

```
models = ["KNN", "Logistic Regression", "Decision Tree"]

accuracies = [
    accuracy_score(y_test, y_pred),
    lr_accuracy,
    dt_accuracy
]

plt.bar(models, accuracies)
plt.title("Model Accuracy Comparison (Bar Chart)")
plt.ylabel("Accuracy")
plt.show()

plt.plot(models, accuracies)
plt.title("Model Accuracy Comparison (Line Chart)")
```

```
    plt.ylabel("Accuracy")
    plt.show()
```

Interpretation